SAY EARLY, before slide 1

- **Skyforge:** "Skyforge is an internal lab platform I built. It isn't a Forward product and it isn't open source. What we do publish is our patches to the open source tools underneath it, and that's the link at the end."
- **Forward:** "I work at Forward Networks, which makes a commercial digital twin. When I say the twin or Forward, I mean that product. Skyforge is separate: it's the lab platform I built."
- **Predict:** "Predict is the feature in Forward's product that takes a proposed config change and computes what the network would do after it, from the collected model, without touching a device."

Appendix (not presented): slides 27 to 34. 27 Predict harness (if asked), 28 eight defects, 29 firewall as one intent, 30 PR comment, 31 patches upstream, 32 fleet40 demo, 33 where AI may act, 34 why not containerlab.

1. What Building a Network Emulation Platform Taught Me About Digital Twins

DELIVERY 00:00–00:30 (30 seconds).

- SAY EARLY. Say these three before anything else. They define the three names the room will hear most.
- Skyforge: "Skyforge is an internal lab platform I built. It isn't a Forward product and it isn't open source. What we do publish is our patches to the open source tools underneath it, and that's the link at the end."
- Forward: "I work at Forward Networks, which makes a commercial digital twin. When I say the twin or Forward, I mean that product. Skyforge is separate: it's the lab platform I built."
- Predict: "Predict is the feature in Forward's product that takes a proposed config change and computes what the network would do after it, from the collected model, without touching a device."
- The interesting part is what happened when I started asking that lab questions about production. The useful distinction is between evidence from executing a particular implementation and evidence computed from a supported representation. One routing change makes that concrete. The point is a change decision, not a contest between products.

Click: static. Advance after the transition.
Transition: First, one puzzle.

2. Safe to apply?

DELIVERY 00:30–01:30 (60 seconds).

- Puzzle before the thesis: a routing change on one router in a small MPLS VPN lab. At click 3, ask for hands: safe to apply?
- Ticket: tidy the export policy on ce-svc, which advertises the services block, 10.20.30.0/24. The current line permits the /24 and anything more specific (le 32). The proposed line drops le 32. It reads like a tidy-up. Pause for hands.
- Not safe. The service is two /25s. Without le 32 the line matches only the /24 itself, so neither /25 is advertised. The BGP session stays up, and the service is gone.

Click: 1 ticket, 2 current, 3 proposed (hands up), 4 not safe, 5 BGP stays up.
Transition: The thesis.

3. An emulator is not a digital twin

DELIVERY 01:30–03:00 (90 seconds).

- Reading the edit did not catch it. There are two other ways, an emulator and a digital twin, and they answer different questions. That is this talk.
- Thesis: an emulator is not a digital twin. Building one is what made that visible to me.
- An emulator runs the real software on virtual hardware: what does this software do? A digital twin is a model built from the collected state of the real network: what would the real network do? Neither is a better version of the other. The question decides the evidence I need.

Click: 1 to 5, the arc of the talk.
Transition: Why I built it.

4. Why I built Skyforge

DELIVERY 03:00–04:25 (85 seconds).

- SAY EARLY: repeat the Skyforge line from slide 1, including "that's the link at the end."
- Sales engineers need a network to show a customer or practice on, quick and disposable. Building one by hand does not scale. So: many people at once, labs spread across hosts, no manual configuration.
- I got the platform wrong twice. containerlab with clabernetes ran Docker inside Docker inside pods. KNE replaced it in March 2026. Then I ran k3s on hosts I rolled myself, and Talos replaced that in July. My early scripts did steps, not results. The fix: describe the result, and let something reconcile it.

Click: 1 to 5, the five cards.
Transition: Here is the pipeline.

5. One declaration, a running network

DELIVERY 04:25–05:10 (45 seconds).

- The only input is the topology YAML, in Git. netlab reads it itself and renders each device's config, and, through our KNE provider patch, the topology KNE needs.
- KNE creates a pod or VM per device, and meshnet wires the links. Protocols still have to converge. Forward is not in this path. It collects afterwards.
- "Deterministic" means the validated authoring path, not the same wall-clock trace. The benefit is a starting point I can inspect when a result surprises me.

Click: 1 YAML, 2 netlab, 3 KNE and meshnet, 4 converge and test, 5 repeatable, 6 Forward collects.
Transition: What a declaration looks like.

6. What the declaration looks like

DELIVERY 05:10–06:45 (95 seconds).

- The template for the eleven-node lab, trimmed. No device configuration in it. It says what the network is: a routing policy, provider edge routers with their protocols, and links with one policy on one session.
- On the right is what netlab renders for one router. I did not type that. netlab chose the prefix-list sequence number and the names.
- The change I examine later is one word: remove the length qualifier. In the rendered config that is exactly one line of diff. Hold on to that line. Both machines get asked about it.

Click: 1 template, 2 rendered config, 3 the diff, 4 nobody typed the device config.
Transition: How big does it get?

7. The nodes weren't the limit. A controller was.

DELIVERY 06:45–08:00 (75 seconds).

- On 9 October I built a two-node AWS island and ran real Arista cEOS on it. 100 pods Ready in 374 seconds. At 250 it stalled at 220, and the nodes were nowhere near their limits. The cEOS operator was killed for memory 26 times and pinned on CPU. It generates a 4096-bit key for every device. With more CPU and memory, 500 of 500 were Ready in 925 seconds.
- Tested to 500 on two nodes. Bigger is planned, not run. The lesson: fix the operator before buying more nodes. Routers don't care about our capacity planning spreadsheet.

Click: 1 measured, 2 limits, 3 cloud bursting.
Transition: Now, what only a lab gives me.

8. What only an emulator gives me

DELIVERY 08:00–08:50 (50 seconds).

- An emulator runs the real software. The BGP session stayed Established while the router withdrew both prefixes. It gives real boot and transition timing for that virtual setup, and it shows API and CLI behaviour. It also exposes what nobody modelled. In my lab, a management default route on every router is why the failure was a timeout, not a refusal.
- That is observed behaviour of a network I built. That is its strength, and what makes it a different kind of thing from a twin.

Click: 1 real software, 2 timing, 3 API and CLI, 4 what nobody modelled.
Transition: It can still be right about the wrong network.

9. The lab can be right about the wrong network

DELIVERY 08:50–09:35 (45 seconds).

- Same edit, different route input. The lab advertises a summary, and production advertises more-specific routes. Both routers can run their config correctly.
- External peers, return paths, current failures and dependencies outside the copied topology work the same way. A passing probe proves the case I exercised. It does not quantify over every source, destination, protocol and state.
- The lab may have a default route production lacks. The HTTPS probe passes and still misleads. A missing return route can make a correct change look broken.

Click: 1 real NOS, 2 outside the lab, 3 a passing test covers the world I built.
Transition: A different question from the one a lab answers.

10. An emulator and a digital twin differ in kind

DELIVERY 09:35–11:00 (85 seconds).

- They are different kinds of thing, not a better and a worse version.
- Emulator: the source of truth is the topology I declare. It runs the actual software, so it answers what this software does, with observed evidence. Each question costs a boot.
- Digital twin: the source of truth is state collected from the real network. It runs a model of supported behaviour, so it answers what the real network would do, with computed evidence. Each question costs a query.
- Back to the prefix-list. A lab can run it. Only a model of the whole network shows which paths it takes away.

Click: seven, one table row each.
Transition: One change, run on both machines.

11. Eleven nodes, one policy boundary

DELIVERY 11:00–12:10 (70 seconds).

- From the branch client through ce-branch and the provider MPLS core to ce-svc and the HTTPS endpoint. Customer branch AS 65010, services site AS 65020, provider AS 65000 in between.
- The candidate export policy is on ce-svc toward pe2. Source 10.10.10.10, destination 10.20.30.10, TCP 443. ce-svc advertises 10.20.30.0/25 and 10.20.30.128/25. No covering /24, and no default route in the customer VRF. Careful with "default": every router has a management default route.
- A probe to one address does not establish the fate of the other /25.

Click: 1 topology, 2 client and service, 3 the two /25s, 4 no covering /24.
Transition: Now connect the declaration to the running network.

12. Skyforge: what I observed

DELIVERY 12:10–13:20 (70 seconds).

- Before capture first: ce-svc advertising two /25s to pe2, session Established. After capture, same commands. The route lookup returns only the management default route, ce-svc advertises zero prefixes, and the session is still Established. An adjacency check would have passed while the service failed.
- The strip is the client test: HTTP 200 baseline, curl exit 28 in the candidate, HTTP 200 restored. Exit 28 is a timeout, not a refusal, because of the management default route. The negative control failed in every phase.
- Excerpts of the raw record.

Click: 1 to 3 the pictures, 4 route lookup explains, 5 HTTPS checks the requirement.
Transition: Now the same baseline and edit in the model.

13. Predict: the evidence I need

DELIVERY 13:20–14:30 (70 seconds).

- SAY EARLY (repeat of the slide 1 Predict line): "Predict is the feature in Forward's product that takes a proposed config change and computes what the network would do after it, from the collected model, without touching a device."
- Base snapshot 13984, from the lab. Change set CHG-752 stages one CLI line on ce-svc. Predicted snapshot 13985 shows that line removed and one added. TCP 443 is delivered on the baseline and dropped at ce-branch in the prediction. Eight of 56 subnet pairs are newly isolated, none newly connected.
- Limits. No checks are attached, so "no check got worse" is not evidence. Not tested, so not established: ECMP, asymmetric route targets, cross-VRF isolation, BGP-LU and SR. Not tested is not the same as unsupported.
- Captured 2026-10-04, not yet matched to the run record. If asked which Forward build, say the 26.9 line, not a build number, until I have.

Click: 1 to 3 the pictures.
Transition: Same intent, different evidence.

14. Same intent, different evidence

DELIVERY 14:30–15:40 (70 seconds).

- Left column is my lab, Skyforge, observed execution. Right is Forward's Predict, modeled behavior. Say that before you read the rows.
- Same intent: branch HTTPS reachability. They agree on the outcome: both /25s gone, return still reachable. They differ in form. The lab showed an HTTPS timeout, because the CE has a management default route. Predict showed an explicit drop at ce-branch.
- Not a benchmark. A path result does not validate server health or production interruption time.

Click: 1 Skyforge, 2 withdrawn, 3 evidence, 4 Predict, 5 path removed, 6 evidence, 7 limits.
Transition: When do I reach for which?

15. I reach for emulation when software matters

DELIVERY 15:40–16:10 (30 seconds).

- My first instinct is emulation when the uncertainty is in software interaction. Does configuration apply, what does the API report, does automation behave after a partial failure. The implementation is the subject.
- It is also useful for protocol transitions and for reproducing a defect, with the relevant software and conditions. A functional reproduction is not a performance claim. The same NOS code in a virtual workload does not reproduce hardware forwarding or timing.

Click: 1 CLI and API, 2 failure handling, 3 protocol interaction, 4 virtual convergence timing does not transfer.
Transition: Other questions send me to the model.

16. I reach for a model when scope matters

DELIVERY 16:10–16:40 (30 seconds).

- I start with the model when the uncertainty is scope: which paths change, where a policy blocks traffic, which requirements are affected across many devices.
- I do not smuggle completeness into the word model. The result is bounded by collected state, supported semantics and the query. Missing inputs mean collection, another method, or an explicit unknown.
- Failure impact is the question I'd put to a model. I am not showing that run, so I won't claim one. Demo 4 is the lab side of it.

Click: 1 reachability, 2 policy, 3 production context, 4 what it depends on.
Transition: That is the twin as a judge. Now a change that waits for it.

17. A change waits until the twin has judged it

DELIVERY 16:40–19:30 (170 seconds).

- The lab proves the machinery works: it applies, holds and undoes the change on the real device. The twin, Forward, judges the change. A twin cannot tell you your undo works.
- The change is in git, with its own checks. napalm pushes it with an automatic undo armed. Forward collects a fresh snapshot and asks three questions. All pass: the undo is cancelled. One fails: the change is undone, and the run says which check failed.
- Predict can check a firewall rule before any device is touched. Proven live once, off by default. Scope: cEOS, IOL and a PA-VM, in labs. Not production.

Click: seven, one per step.
Transition: So what does Forward ask?

18. Forward asks three things

DELIVERY 19:30–22:10 (160 seconds).

- One: did anything I was running disappear? Forward compares BGP sessions, LLDP neighbours and interface state before and after. On the ring demo, BGP sessions were 88 before and 88 after.
- Two: did anything else break? Forward re-runs every existing check on both snapshots. Only a check that passed before and fails now counts. On the firewall fix, 118 of 118.
- Three: did the change do its job? The change names its own checks: who must reach what, what must stay isolated.
- A check that looks at nothing is a failure, not a pass.

Click: 1 disappear, 2 break, 3 job, 4 the fix card (run 2aa18f9f).
Transition: Now watch it happen.

19. The wrong half of the range, held and undone

DELIVERY 22:10–28:50 (400 seconds).

- PLAY THE CLIP (68 seconds). Break run b6f0435e, fix run 2aa18f9f. HELD 4 to 11 s: the planted digit, undone platform-side because PAN-OS has no commit-confirm. REVERTED 11 to 18 s: snapshot 14840 restored, 14841 after. Evidence 18 to 25 s: 118 checks, 2 regressions, pd3_fwin_h_to_fwout_h pass to fail. Break PR comment 25 to 32 s. Fix HELD 32 to 38 s. CONFIRMED 38 to 45 s, 14844 to 14845. Evidence 45 to 51 s. Fix PR comment 51 to 58 s. Close 58 to 68 s.
- Dry aside, if it fits: the firewall did exactly what I told it. That was the problem.

Click: play the clip.
Transition: Firewalls are different.

20. A firewall can’t undo itself, so the platform does

DELIVERY 28:50–31:05 (135 seconds).

- First, the undo. Arista EOS and Cisco IOS roll back on their own. PAN-OS has no commit-confirm, so the platform saves a named config and a reaper restores it past a deadline. A platform-side revert, not a device timer, and weaker: if the change cut management access, the platform could not reach the firewall.
- Second, what Predict reads. PAN-OS security rules, not CLI set lines. One structured rule is rendered two ways, so the prediction and the set lines can't drift.
- Pre-flight, proven live on 2026-10-10, run b9a03a53. Held before any device was touched. Off by default.

Click: 1 to 4, one per card.
Transition: How I'd like to write changes.

21. Say the change once: built, run on one change

DELIVERY 31:05–33:00 (115 seconds).

- Intent is what the network should do, in a form a machine can check. If I can't write the check, it's a wish.
- In use: the deployment pack, where each change states its device config, Forward checks and Predict probes. They can drift.
- Built, run once: one vendor-neutral intent. The live run, 394f25f2-4260-45c2-a576-3492c420bf89, applied the good version, and Forward confirmed it, 118 of 118. The shared pack still uses the old form. A prototype matched 59 of 60 comparisons on six fleet40 changes.
- If asked: one change, once.

Click: today, written once, raw CLI marked, built and run once.
Transition: What went wrong on the first real firewall.

22. The twin judged it right. The machinery broke.

DELIVERY 33:00–35:20 (140 seconds).

- The twin judged the change. What broke was the machinery.
- Eight bugs, and our test fake could not raise any of them. Four of them: PAN-OS refuses a saved-config name over 32 characters. "Save config" saves the candidate, not the running config, so a leftover became the rollback point of every later change. The CLI library raised an error on a commit that had worked. And "application any" with "application-default" is rejected at commit.
- The lesson: a fake tells you your code does what you think. A firewall tells you what it does, and it has opinions.

Click: 1 names, 2 save, 3 library, 4 rule, 5 the lesson.
Transition: Where AI helps.

23. AI helps. A person still approves.

DELIVERY 35:20–37:00 (100 seconds).

- Yes, I used AI to build this. But the platform's operations are deterministic, so I want to be exact about where AI sits.
- Working today: AIOps triages incidents and drafts bounded fixes. A person approves. Autonomous execution is off.
- Run live, once: on 2026-10-11 the loop ran on the demo firewall. A model drafted the rework, a person approved it, Forward confirmed it. The loop never approves or applies anything itself. One lab and one change: a proof, not a track record.
- Forward and Predict never push to devices.

Click: 1 working today, 2 run live once, 3 never push.
Transition: What I take away.

24. What I took away

DELIVERY 37:00–38:15 (75 seconds).

- Keep the two questions apart. The lab tells me what this software does. The twin tells me what happens in my network. I don't swap them.
- On change control I need all three. The lab proves the machinery works. The checks and regression sweep gate the change. The twin judges whether it does what the network needs. The first real firewall run found eight bugs in the machinery, and the test fake could not raise any of them.
- Last one: a check that looks at nothing is a failure, not a pass.

Click: 1 to 5, one per line above.
Transition: The question I ask now.

25. The question I ask now

DELIVERY 38:15–39:00 (45 seconds).

- What am I trying to learn, and what evidence would change my decision? If the risk is a NOS quirk, I want the lab. If it's a broad path or policy change, I want the twin. If a change is going onto a real device, I want both.
- End on that question and pause.
- The QR code goes to github.com/forwardnetworks/skyforge. It holds only our patches to upstream open source projects. Skyforge itself is not in it, and Skyforge is not open source. Do not spend time on it.

Click: 1 "What am I trying to learn?", 2 "What evidence would change my decision?"
Transition: Questions.

26. Questions

DELIVERY 39:00–45:00 (360 seconds).

- Open Q&A. I'd like to hear where a lab result misled someone, what was missing, and which evidence eventually explained it.
- Prompts if needed: How do you record snapshot freshness? Which features still need hardware? What makes an unknown acceptable for a change?
- Stay in scope. This is not a benchmark: one lab, one change, run twice on 2026-10-04, and the model covers only supported behavior.

Click: 1 "Where has your lab given you a misleading answer?"